



1

(3 marks)

---

Write a C program to check whether a given square matrix is equal to its transpose (i.e., symmetric matrix).

2/0

(5 marks)

Write a C programming function, which take a square matrix (as a 2D array) as input and checks if it is idempotent (ie.  $A^2 = A$ ). It should return 1 if the idempotent condition is satisfied, otherwise zero.

✓  
3

(3 marks)

---

Write a c code for sorting an array of integers.

4

(3 marks)

Consider the following recursive function  $\text{fun}(x, y)$ .  
What is the value of  $\text{fun}(4, 3)$ ?

```
1  int fun(int x, int y) {  
2      if (x == 0)    return y;  
3      return fun(x - 1, x + y);  
4  }
```

✓5

(5 marks)

What will the code below print if we call fun(4)?

```
1 void fun(int x) {  
2     if(x > 0) {  
3         fun(--x);  
4         printf("%d \t ", x);  
5         fun(--x);  
6     }  
7 }
```

Fill in the missing line so that the program prints the sum of all diagonal elements of a  $3 \times 3$  matrix.

```
1  #include <stdio.h>
2  #define N 3
3
4  int main() {
5      int arr[N][N] = { {1,2,3},
6                          {4,5,6},
7                          {7,8,9} };
8      int sum = 0, i;
9
10     for(i=0; i<N; i++) {
11         // missing line here
12         }
13
14     printf("Sum = %d \n", sum);
15     return 0;
16 }
17
```

*Sum += arr[i][i];*



7/10

(3 marks)

The Ackermann's function is defined as follows:

$$\varphi(m, n, 0) = m + n$$

$$\varphi(m, 0, 1) = 0$$

$$\varphi(m, 0, 2) = 1$$

$$\varphi(m, 0, p) = m \quad \text{for } p > 2$$

$$\varphi(m, n, p) = \varphi(m, \varphi(m, n - 1, p), p - 1) \quad \text{for } n, p > 0$$

Write a C function, which computes it.

~~7~~ ~~b~~

---

(4 marks)

Show the tree of recursive calls to the C function written above, for computing  $\varphi(2, 2, 1)$ .



Read the code and say exactly what it prints. Basically you just have to print the final output.

```
1  #include <stdio.h>
2
3  int mystery(int n) {
4      if (n <= 1) return n;
5      if (n % 2 == 0) return mystery(n / 2);
6      return mystery(n / 2) +
7          mystery(n / 2 + 1);
8  }
9
10 int main() {
11     printf("%d \n", mystery(1));
12     printf("%d \n", mystery(2));
13     printf("%d \n", mystery(3));
14     printf("%d \n", mystery(4));
15     printf("%d \n", mystery(5));
16     printf("%d \n", mystery(6));
17     return 0;
18 }
```

Complete the blank so that the array is rotated to the right by one position. For example, if the input array is [2, 3, 4, 5], the output should be [5, 2, 3, 4]. Only a single statement is permitted.

```
1  int main() {  
2      int a[4] = {1,2,3,4};  
3      int tmp = a[3];  
4      /* === FILL === */  
5      a[0] = tmp;  
6      for (int i = 0; i < 4; i++)  
7          printf("%d", a[i]);  
8      return 0;  
9  }
```

Given an array  $\text{arr}[]$  of size  $n$ , the task is to rearrange it in alternate positive and negative manner without changing the relative order of positive and negative numbers. In case of extra positive/negative numbers, they appear at the end of the array.

Example 1:

Input:  $\text{arr}[] = \{1, 2, 3, -4, -1, 4\}$

Output:  $\{1, -4, 2, -1, 3, 4\}$

Example 2

Input:  $\text{arr}[] = \{-5, -2, 5, 2, 4, 7, 1, 8, 0, -8\}$

Output:  $\{-5, 5, -2, 2, -8, 4, 7, 1, 8, 0\}$